

Overview

Track 16.5 — 🚀 LangSmith · 16-ai-tech-stack/tracks/langsmith/README.md

🚀 Track 16.5 — LangSmith: Tracing, Evals & Observability

Where you are: the observability and evaluation layer of the AI tech stack (Module 16). **Time:** ~8–10 hours · **Prereq:** Python, one LLM app you have actually shipped or want to, and ideally a skim of `../15-ai-evals/`.

LLM applications break while returning HTTP 200. There is no stack trace for "the answer got worse", no failing test for a prompt edit that quietly cost you 12% accuracy on one customer segment, and no alert for the retriever change that doubled your bill. The trace is the only debugger you get, and the loop `trace → find failure → add to dataset → fix → eval → ship → monitor` is the only thing that makes an LLM app improve instead of drift. LangSmith is the most complete implementation of that loop; this track teaches the loop first and the vendor second.

LangSmith is a hosted service, so **every script here runs offline with no account and no API key** — you rebuild its data model yourself, which is a better way to learn it than clicking through a free trial.

Learning objectives

By the end of this track you can:

1. Explain why LLM failures are invisible to conventional logging, metrics and tests — with a named example of each of the four failure modes (silent regression, cost drift, nondeterminism, no stack trace).
2. Instrument an arbitrary Python app — LangChain or not — with `@traceable`, client wrappers, manual run trees and the right environment variables, and debug a bad answer from its run tree alone.
3. Design the sampling and PII-redaction policy for a production tracing deployment, and defend both choices to a security reviewer.
4. Build an evaluation dataset by harvesting production traces, with splits, versions and a pinned CI tag.
5. Write evaluators against the `evaluate()` contract — deterministic, rubric-based judge, pairwise — and name the three biases that make an uncalibrated judge worthless.
6. Wire a CI gate with both absolute floors and regression deltas against a stored baseline, and explain why you need both.
7. Choose between LangSmith, Langfuse, Phoenix, Braintrust, raw OpenTelemetry and plain logging for a given team, and say out loud what you are giving up.

Suggested path

#	Do this	File	Time
1	Read the tracing data model	notes/01-core-concepts.md	1.5h
2	Rebuild the run tree yourself	code/minimal_tracer.py	1h
3	Read evaluation & operations	notes/02-evaluation-and-operations.md	1.5h
4	Run the offline eval harness	code/eval_harness.py	1h
5	Read the real SDK, annotated	code/langsmith_instrumentation_reference.py	0.5h
6	Present it back	slides/(langsmith.pptx)	0.5h
7	Do the exercises	lab/EXERCISES.md	3h
8	Read the primary docs	papers/PAPERS.md	1h

```
cd code
python minimal_tracer.py          # run trees, cost rollups, redaction
python eval_harness.py           # judges, pairwise, CI gate
python eval_harness.py --strict   # the CI-mode exit code
python langsmith_instrumentation_reference.py  # the real SDK, no key needed
```

The 16 terms you must own

run · trace · run tree · span · run_type · tracing project · thread · tag · metadata · feedback · dataset · example · split · experiment · LLM-as-judge · annotation queue

Exit check

Take any LLM app you have — even a 30-line script — and produce:

1. A traced run tree with correct `run_type`s, a release SHA in metadata, and `thread_id` on multi-turn calls.
2. A dataset of **≥ 10 examples, every one harvested from a real trace**, with a `smoke` split.
3. Three evaluators: one deterministic, one rubric-style judge, one budget check (latency or cost).
4. A baseline JSON and a CI gate that **fails** when you deliberately regress the prompt — and passes when you revert it.
5. One paragraph naming the bias most likely to fool your judge, and what you did about it.

If your CI gate cannot catch a regression you introduced on purpose, it will not catch one you introduced by accident.